# Lab 14 – Web Frontend Development

Course Title: AI Assisted Coding
Assignment Number: 14.4
2303A51399

---

# Task 1: Responsive Homepage Layout for a Startup Website

## Question

Design a responsive homepage for a startup company.
The page must include:

- Header with navigation links
- Main content section
- Footer with company information

Use AI assistance to generate responsive CSS using media queries so that the layout works properly on desktop, tablet, and mobile screens.

---

## AI Prompt Used

Write HTML and responsive CSS using media queries for a startup homepage with header navigation, main content section, and footer. Ensure the layout adapts for desktop, tablet, and mobile devices.

---

## Code

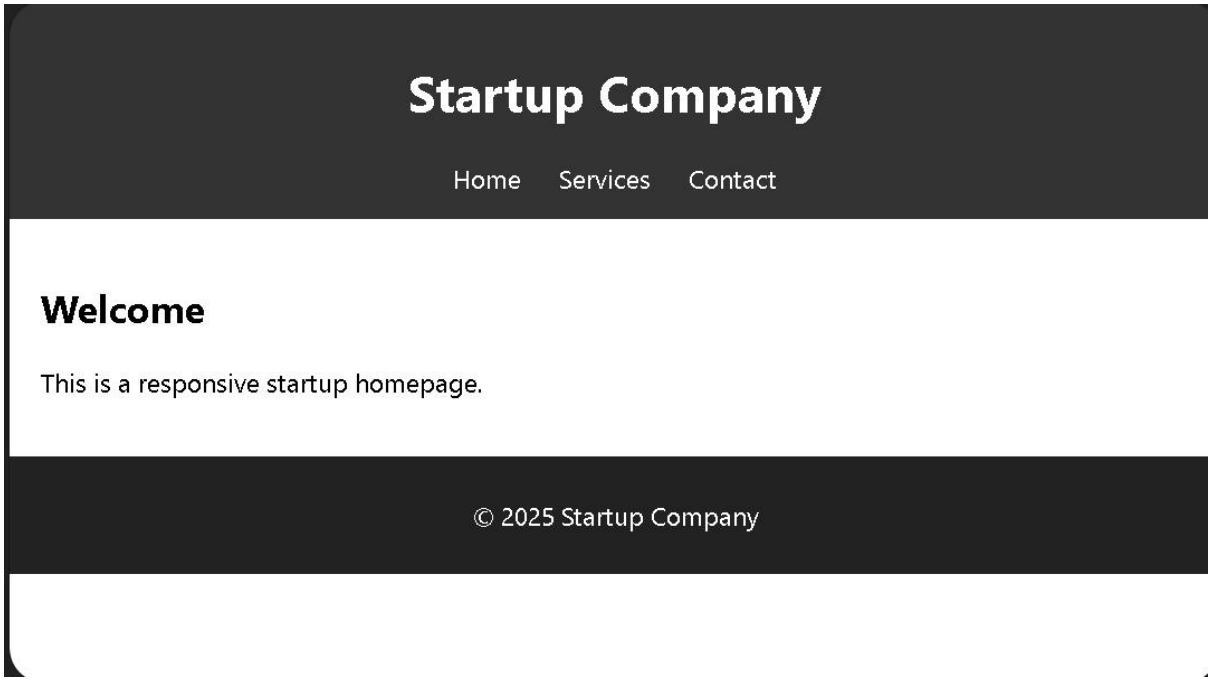
```
<!DOCTYPE html>
<html>
<head>
<title>Startup Homepage</title>

<style>
```

```
/* Basic page styling */
body{
font-family: Arial;
margin:0;
}

/* Header styling */
header{
background:#333;
color:white;
padding:15px;
text-align:center;
}

/* Navigation links */
nav a{
color:white;
margin:10px;
text-decoration:none;
}

/* Main content */
main{
padding:20px;
}

/* Footer */
footer{
background:#222;
color:white;
text-align:center;
padding:10px;
}

/* Tablet view */
@media(max-width:768px){
nav{
display:block;
text-align:center;
}
}

/* Mobile view */
@media(max-width:480px){
header,footer{
font-size:14px;
}
}
```

```
</style>
</head>

<body>

<header>
<h1>Startup Company</h1>
<nav>
<a href="#">Home</a>
<a href="#">Services</a>
<a href="#">Contact</a>
</nav>
</header>

<main>
<h2>Welcome</h2>
<p>This is a responsive startup homepage.</p>
</main>

<footer>
<p>© 2025 Startup Company</p>
</footer>

</body>
</html>
```

---

# Output




A responsive web page where:

- Header displays navigation links
- Main content appears in the center
- Footer shows company information
- Layout adjusts for mobile, tablet, and desktop screens

---

## Code Explanation

The HTML structure defines three main sections: header, main content, and footer.
CSS styling controls layout and visual appearance.
Media queries adjust the layout for different screen sizes.
This ensures that the page remains readable and usable across multiple devices.

---

# Task 2: Interactive Call-to-Action Button

## Question

Create a call-to-action button that responds when users click it.
Use JavaScript to display a message without refreshing the page.

---

# AI Prompt Used

Generate JavaScript code that triggers an alert when a button is clicked and include comments explaining the logic.

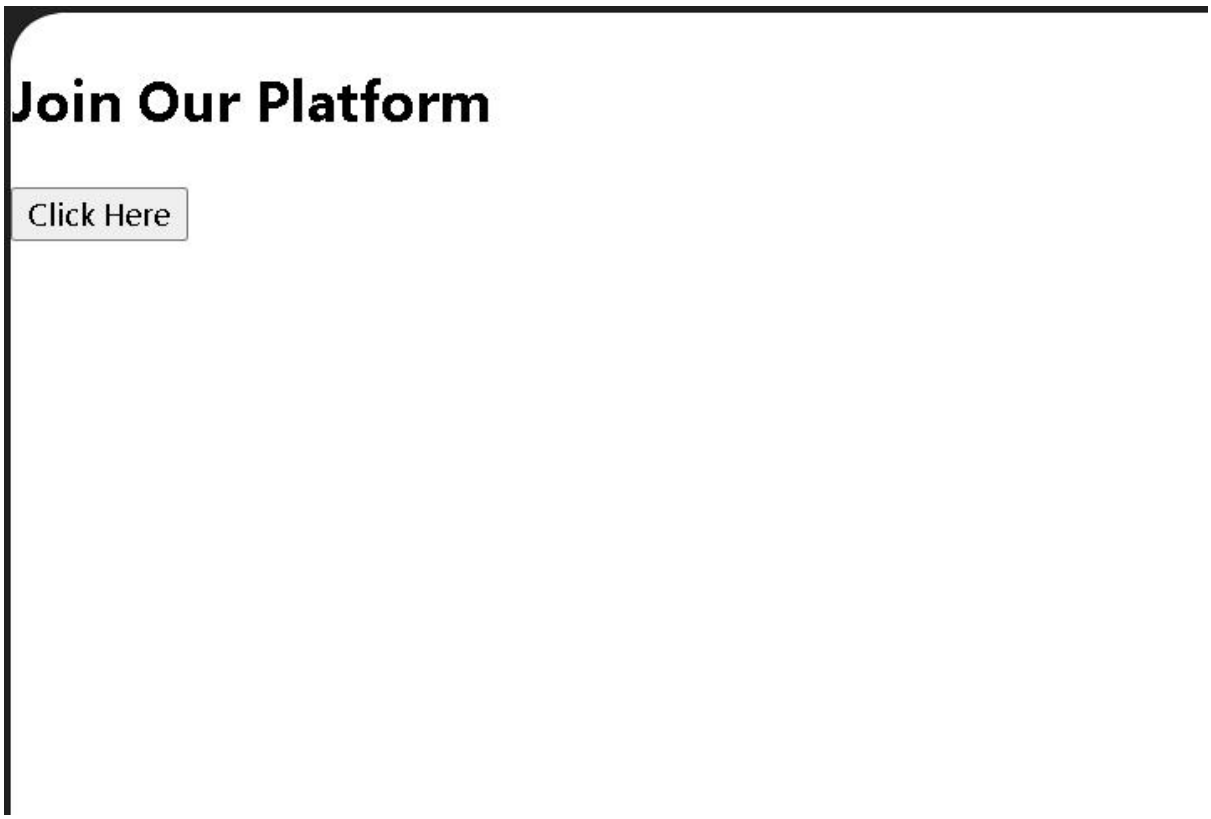
---

# Code

```
<!DOCTYPE html>
<html>
<head>
<title>CTA Button</title>

<script>

// Function triggered when button is clicked
function showMessage(){

// Display alert message
alert("Thank you for your interest in our startup!");

}

</script>
</head>

<body>

<h2>Join Our Platform</h2>

<button onclick="showMessage()">Click Here</button>

</body>
</html>
```

---

# Output




When the user clicks the button, an alert message appears saying:

“Thank you for your interest in our startup!”

---

## Code Explanation

The HTML button triggers the JavaScript function *showMessage()* using the *onclick* event.

The function executes immediately and displays an alert message.
Since the script runs on the client side, the page does not reload.

---

# Task 3: Contact Form with Client-Side Validation

## Question

Create a contact form with the following fields:

- Name

- Email
- Message

Use JavaScript to validate input before submission.

---

# AI Prompt Used

Generate JavaScript validation for a contact form that prevents empty inputs and checks valid email format.

---

# Code

```
<!DOCTYPE html>
<html>
<head>
<title>Contact Form</title>

<script>

function validateForm(){

var name=document.getElementById("name").value;
var email=document.getElementById("email").value;
var message=document.getElementById("message").value;

if(name=="" || email=="" || message==""){
alert("All fields are required");
return false;
}

/* Email format validation */
var emailPattern=/^\S+@\S+\.\S+$/;

if(!emailPattern.test(email)){
alert("Enter a valid email");
return false;
}

alert("Form submitted successfully");
return true;

}
```

```
</script>

</head>

<body>

<form onsubmit="return validateForm()">

Name:<br>
<input type="text" id="name"><br><br>

Email:<br>
<input type="text" id="email"><br><br>

Message:<br>
<textarea id="message"></textarea><br><br>

<input type="submit" value="Submit">

</form>

</body>
</html>
```

---

# Output


- Empty fields trigger an error message
- Invalid email format triggers validation warning
- Valid input displays success alert

---

## Code Explanation

The form uses JavaScript validation before submission.
The script checks if any field is empty.
A regular expression verifies the email format.
If validation fails, the form submission is prevented.

---

# Task 4: Dynamic List Management

# Question

Create a product list where items can be dynamically added or removed without refreshing the page.

---

# AI Prompt Used

Generate JavaScript code to dynamically add and remove items from an HTML list using DOM manipulation.

---

# Code

```
<!DOCTYPE html>
<html>
<head>
<title>Dynamic List</title>

<script>

function addItem(){

var list=document.getElementById("list");
var newItem=document.createElement("li");

newItem.textContent="New Product";

list.appendChild(newItem);

}

function removeItem(){

var list=document.getElementById("list");

if(list.lastChild){
list.removeChild(list.lastChild);
}

}

</script>

</head>
```

```
<body>

<h2>Product List</h2>

<ul id="list">
<li>Product 1</li>
<li>Product 2</li>
</ul>

<button onclick="addItem()">Add Item</button>
<button onclick="removeItem()">Remove Item</button>

</body>
</html>
```

---

## Output


- Clicking **Add Item** inserts a new list element
- Clicking **Remove Item** deletes the last element
- Changes appear immediately without page refresh

---

## Code Explanation

JavaScript manipulates the DOM using `createElement()` and `appendChild()` to add items.

The `removeChild()` method removes elements dynamically.

This allows real-time updates without refreshing the page.

---

# Task 5: Modal Popup for User Notifications

## Question

Create a modal popup with:

- Overlay background
- Open and close functionality
- Multiple closing options

---

## AI Prompt Used

Generate HTML, CSS, and JavaScript code for a modal popup with overlay and close functionality.

---

## Code

```
<!DOCTYPE html>
<html>
<head>
<title>Modal Popup</title>

<style>

.modal{
display:none;
position:fixed;
top:0;
left:0;
width:100%;
height:100%;
background:rgba(0,0,0,0.5);
```

```
}

.modal-content{
background:white;
padding:20px;
margin:15% auto;
width:300px;
text-align:center;
}

</style>

<script>

function openModal(){
document.getElementById("modal").style.display="block";
}

function closeModal(){
document.getElementById("modal").style.display="none";
}

</script>

</head>

<body>

<button onclick="openModal()">Open Modal</button>

<div id="modal" class="modal" onclick="closeModal()">

<div class="modal-content">

<p>This is a notification popup</p>

<button onclick="closeModal()">Close</button>

</div>

</div>

</body>
</html>
```

---

# Output


- Clicking **Open Modal** displays the popup
- Popup appears above page content with dark overlay
- Clicking **Close** or the overlay hides the modal

---

# Code Explanation

The modal popup is hidden by default using *display:none*.

JavaScript functions control visibility by changing the display property.
CSS overlay provides visual focus on the popup content.